HDOC Day-47

****

Write a C program that reads the size and elements of a sorted variable-length integer array, arr[], in ascending order. Also, read an integer target. Pass the array to a function named ceil_index() using its address. Use pointers to pass the array between the calling function and ceil_index(). The function ceil_index() must:

- Find the index of the smallest element in arr[] that is >= target.
- Treat this element as the ceil of target.
- If multiple occurrences of the ceil element exist, return the index of its first occurrence.
- Display the ceil element and its index.
- Display -1 if no element is >= target.

1. Problem Understanding

The program reads the size of a sorted integer array, its elements, and a target value. The array is passed to the function ceil_index() using a pointer. The function searches for the smallest array element that is greater than or equal to the target. If that element occurs more than once, the index of its first occurrence is returned. If no such element exists, the function returns -1.

2. Standard Test Cases

Test Case	Size (n)	Array Input	Target	Expected Output
1	6	5 7 7 8 8 10	6	Ceil of 6: 7 at index 1
2	6	5 7 7 8 9 10	8	Ceil of 8: 8 at index 3
3	6	5 7 7 8 8 10	11	-1
4	5	2 4 6 8 10	5	Ceil of 5: 6 at index 2
5	5	3 3 3 5 7	3	Ceil of 3: 3 at index 0

3. Algorithm

Main Algorithm

1. Start.
2. Read the size of the array.
3. Declare a variable-length integer array of the given size.
4. Read the sorted array elements in ascending order.
5. Read the target value.
6. Pass the array address, size, and target to ceil_index().
7. If the returned index is -1, display -1.
8. Otherwise, display the ceil element and its index.
9. Stop.

Function: ceil_index()

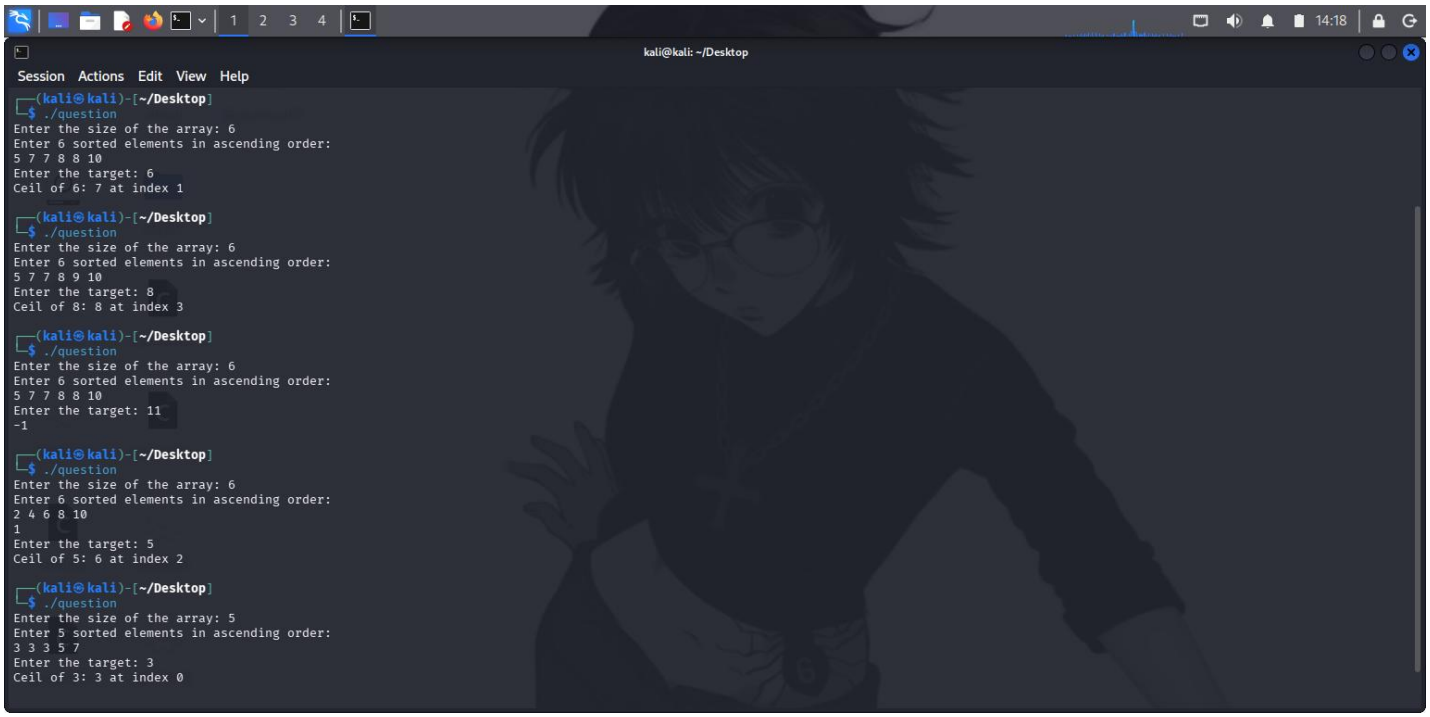
1. Receive the array through an integer pointer.
2. Traverse the array from index 0 to size - 1.
3. For each element, check whether it is greater than or equal to target.
4. Return the first index that satisfies the condition.
5. If no element satisfies the condition, return -1.

4. C Program

```
#include <stdio.h>
int ceil_index(int *arr, int size, int target)
{
   int i;
   for(i = 0; i < size; i++)
   {
      if(*(arr + i) >= target)
      {
         return i;
      }
   }
   return -1;
}
int main()
{
   int size, i, target, index;
   printf("Enter the size of the array: ");
   scanf("%d", &size);
   int arr[size];
   printf("Enter %d sorted elements in ascending order:\n", size);
   for(i = 0; i < size; i++)
   {
      scanf("%d", &arr[i]);
   }
   printf("Enter the target: ");
   scanf("%d", &target);
   index = ceil_index(arr, size, target);
   if(index == -1)
   {
      printf("-1\n");
   }
   else
   {
      printf("Ceil of %d: %d at index %d\n",
            target, arr[index], index);
```

```
    }
    return 0;
}
```

5. Compilation and Execution Evidence



6. Verification of All Test Cases

Test Case	Size (n)	Array Input	Target	Actual Result	Status
1	6	5 7 7 8 8 10	6	Ceil of 6: 7 at index 1	Pass
2	6	5 7 7 8 9 10	8	Ceil of 8: 8 at index 3	Pass
3	6	5 7 7 8 8 10	11	-1	Pass
4	5	2 4 6 8 10	5	Ceil of 5: 6 at index 2	Pass
5	5	3 3 3 5 7	3	Ceil of 3: 3 at index 0	Pass